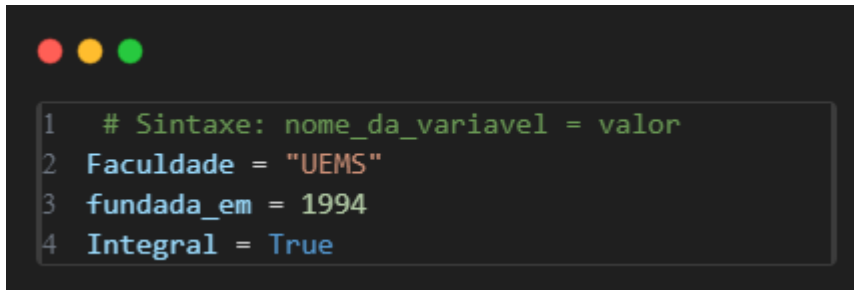


Curso python - Aula 2

1. Variáveis - A Memória do seu Programa

Na programação em Python, as variáveis se destacam por sua simplicidade e flexibilidade, uma consequência direta da natureza dinâmica da linguagem. Diferente de muitas outras linguagens de programação, em Python, as variáveis não são declaradas explicitamente com um tipo antes de serem utilizadas. Em vez disso, elas são criadas no momento em que um valor é atribuído a elas pela primeira vez.

Pense em uma variável não apenas como uma "caixa", mas como um rótulo ou um atalho que aponta para um local na memória do computador onde um dado está armazenado. Quando você cria uma variável, o Python reserva um espaço na memória e associa o nome que você escolheu a esse espaço. O sinal de atribuição (=) é o operador que usamos para gravar um dado em um espaço da memória que está associado àquele rótulo.



```
1  # Sintaxe: nome_da_variavel = valor
2  Faculdade = "UEMS"
3  fundada_em = 1994
4  Integral = True
```

Ao nomear variáveis em Python, é preciso seguir algumas regras e convenções:

- Devem começar com uma letra (a-z, A-Z) ou um sublinhado (`_`).
- O restante do nome pode conter letras, números (0-9) e sublinhados.
- Não podem ser uma das palavras-chave reservadas do Python (como `if`, `for`, `while`, `def`, etc.).

2.Aprofundando nos Tipos de Dados

Python é uma linguagem de tipagem dinâmica, o que significa que você não precisa declarar o tipo de uma variável. O próprio Python descobre o tipo com base no valor que você atribui. Os tipos fundamentais são:

str (texto)

int (inteiros)

float (decimais)

bool (verdadeiro/falso).

A característica principal do sistema de **tipagem dinâmica** é permitir que o tipo de uma variável não seja fixo e possa ser determinado em tempo de execução, com base no valor que ela armazena. Uma mesma variável pode, ao longo de um programa, referenciar um número inteiro, depois uma string (texto) e, em seguida, uma lista, sem a necessidade de qualquer declaração prévia de tipo.

Você pode usar a função `type()` para descobrir o tipo de qualquer variável.

```
1 nome_do_curso = "Ciências da Computação"
2 ano = 2025
3 conclusao = 0.65
4 Integral = True
5
6 print(type(nome_do_curso))
  # Saída: <class 'str'>
7 print(type(ano))
  # Saída: <class 'int'>
8 print(type(conclusao))
  # Saída: <class 'float'>
9 print(type(Integral))
  # Saída: <class 'bool'>
```

```
1 Faculdade = "UEMS"
2 foi_fundada = 1994 # inteiro
3 foi_fundada = "1994" # string
4 foi_fundada = True # booleano
```

Mas é sempre bom se lembrar que para um programa bem estruturado há a necessidade de se manter um padrão ao longo do desenvolvimento, portanto armazenar diferentes tipos em uma variável pode ser considerado uma má prática de programação.

3.Estrutura e indentação em python

Diferente de muitas outras linguagens de programação que usam chaves `{ }` para delimitar blocos de código, o Python usa o **espaçamento em branco** no início de uma linha. Esse espaçamento é chamado de **indentação** .

Em Python, a indentação não é apenas uma questão de estilo para deixar o código mais legível; **ela é uma regra da sintaxe da linguagem**. É a forma como o interpretador Python sabe quais linhas de código pertencem a uma determinada estrutura, como um loop `for`, uma condição `if`, uma função `def`, ou uma classe `class`.

Sem a indentação correta não é possível executar um código em python, mas isso é um dos diferenciais em python que o torna uma linguagem mais curta que as demais e mais intuitiva.

Como Escrever e Como Funciona a Indentação

A regra é simples: todo bloco de código que está logicamente "dentro" de outro comando deve ser indentado.

3.1. A Regra do "Dois Pontos" (:)

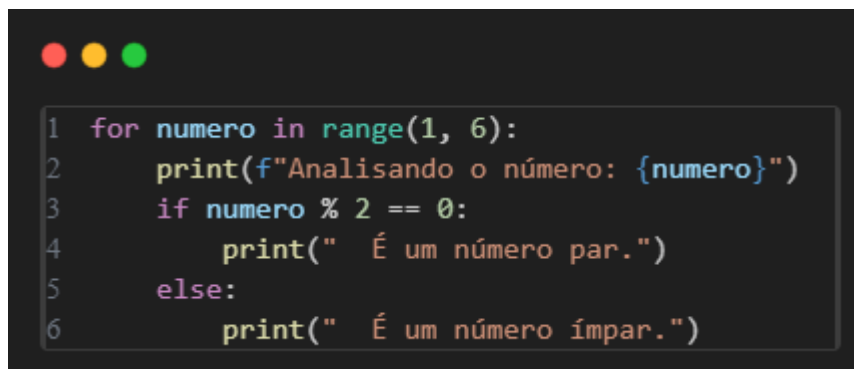
Um bloco de código indentado quase sempre começa depois de uma linha que termina com dois pontos (:). Esses dois pontos sinalizam ao Python: "Atenção, o que vem a seguir é um bloco de código que pertence a este comando".

3.2. O Padrão: 4 Espaços

A convenção oficial do Python (conhecida como **PEP 8**) recomenda o uso de **4 espaços** para cada nível de indentação. Embora você possa usar tabs, o uso de espaços é o padrão da indústria e evita problemas de formatação entre diferentes editores de texto. A maioria dos editores de código modernos pode ser configurada para inserir 4 espaços automaticamente quando você pressiona a tecla **Tab**. por padrão no vscode o tab já tem este espaçamento configurado.

3.3. Como Funciona na Prática

Vamos ver um exemplo claro.

A screenshot of a code editor window with a dark background. The code is written in Python and demonstrates indentation using 4 spaces. The code is as follows:

```
1 for numero in range(1, 6):
2     print(f"Analisando o número: {numero}")
3     if numero % 2 == 0:
4         print(" É um número par.")
5     else:
6         print(" É um número ímpar.")
```

The code is numbered 1 through 6 on the left side of the editor. The indentation is clearly visible, with each level of nesting (the loop body, the if statement, and the print statements) indented by 4 spaces.

O Python sabe que os comandos **if** e **else** pertencem ao loop **for** porque estão indentados em relação a ele. E os **prints** internos pertencem ao **if/else** por estarem indentados em relação a eles.

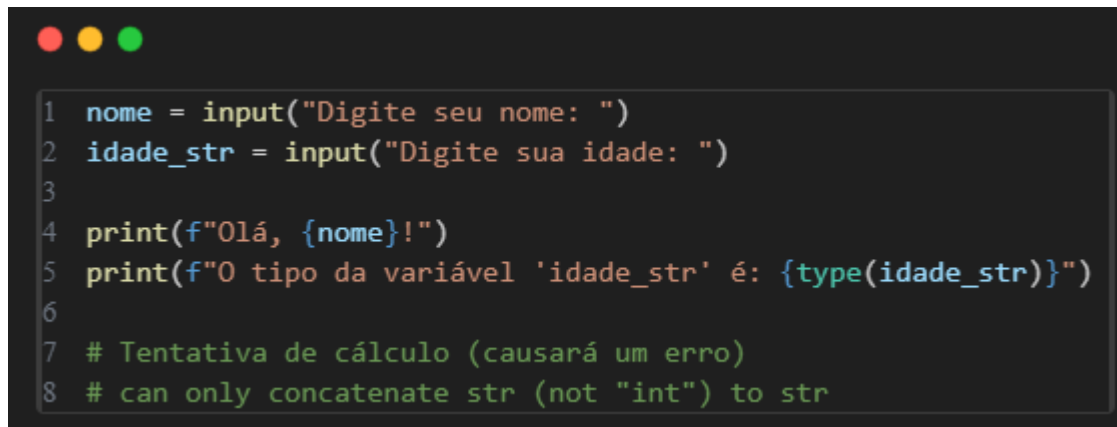
O Erro Mais Comum: `IndentationError`

Se você esquecer de indentar, ou se a sua indentação for inconsistente (por exemplo, usar 3 espaços em uma linha e 4 em outra dentro do mesmo bloco), o Python irá gerar um erro.

4. Entrada de dados: `input()`

A função `input()` tem um propósito simples: pausar a execução do programa, exibir uma mensagem opcional para o usuário (chamada de *prompt*) e aguardar que ele digite algo e pressione Enter.

O Ponto Mais Importante: A função `input()` **sempre retorna uma string (`str`)**. Isso significa que, mesmo que o usuário digite um número, o valor armazenado na variável será um texto.



```
1 nome = input("Digite seu nome: ")
2 idade_str = input("Digite sua idade: ")
3
4 print(f"Olá, {nome}!")
5 print(f"O tipo da variável 'idade_str' é: {type(idade_str)}")
6
7 # Tentativa de cálculo (causará um erro)
8 # can only concatenate str (not "int") to str
```

Para usar a entrada como um número, você precisa **converter o tipo** explicitamente:

```
1 idade_int = int(input("Digite sua idade: "))
2 altura_float = float(input("Digite sua altura (ex: 1.75): "))
3
4 print(f"Daqui a 5 anos, você terá {idade_int + 5} anos.")
```

5. Saída de dados: (print)

A função `print()` é surpreendentemente versátil. Além de simplesmente exibir texto, ela permite um controle fino sobre como a saída é formatada.

Parâmetros Principais:

- Você pode passar um ou mais objetos (variáveis, textos, etc.) para serem impressos.
- `sep=' '`: Define o caractere que separará múltiplos objetos. O padrão é um espaço.
- `end='\n'`: Define o caractere que será impresso no final. O padrão é uma nova linha (`\n`), que move o cursor para a próxima linha.

```
1 print("Python", "é", "incrível", sep="-")
  # Saída: Python-é-incrível
2 print("Primeira linha", end=" | ")
3 print("Segunda linha")
  # Saída: Primeira linha | Segunda linha
```

5.1. Técnicas de Formatação de Strings

1º Método **f-Strings** - O Método Moderno (Recomendado)

Introduzidas no Python 3.6, as f-strings são a forma mais rápida, legível e conveniente de formatar strings.

Sintaxe: Coloque a letra **f** antes das aspas de abertura. As variáveis ou expressões são colocadas diretamente dentro de chaves

```
1 nome = "Maria"
2 idade = 30
3 saldo = 1234.5678
4
5 print(f"Usuária: {nome}, Idade: {idade} anos.")
6 # Saída: Usuária: Maria, Idade: 30 anos.
7
8 # Você pode colocar expressões diretamente
9 print(f"O dobro da idade é {idade * 2}.")
10 # Saída: O dobro da idade é 60.
11
12 # Controle de formatação (muito poderoso)
13 # :.2f -> formata um float para ter 2 casas decimais
14 print(f"Saldo em conta: R$ {saldo:.2f}")
15 # Saída: Saldo em conta: R$ 1234.57
```


2º Método `str.format()` - Flexível e Compatível

Antes das f-strings, este era o método preferido. Ainda é muito útil, especialmente em situações onde a string de formatação é criada separadamente dos dados.

Sintaxe: Use chaves `{}` como marcadores de posição na string e chame o método `.format()` no final.

```
1 nome = "João"
2 idade = 25
3
4 # Por posição
5 print("O nome é {} e a idade é {}".format(nome, idade))
6
7 # Por índice
8 print("A idade é {1}, o nome é {0}".format(nome, idade))
9
10 # Por palavra-chave
11 print("O nome é {n} e a idade é {i}".format(n=nome, i=idade))
12
13 # Com formatação de casas decimais
14 print("Valor: {:.2f}".format(99.5)) # Saída: Valor: 99.50
```

3º Método **operador %** - E estilo clássico

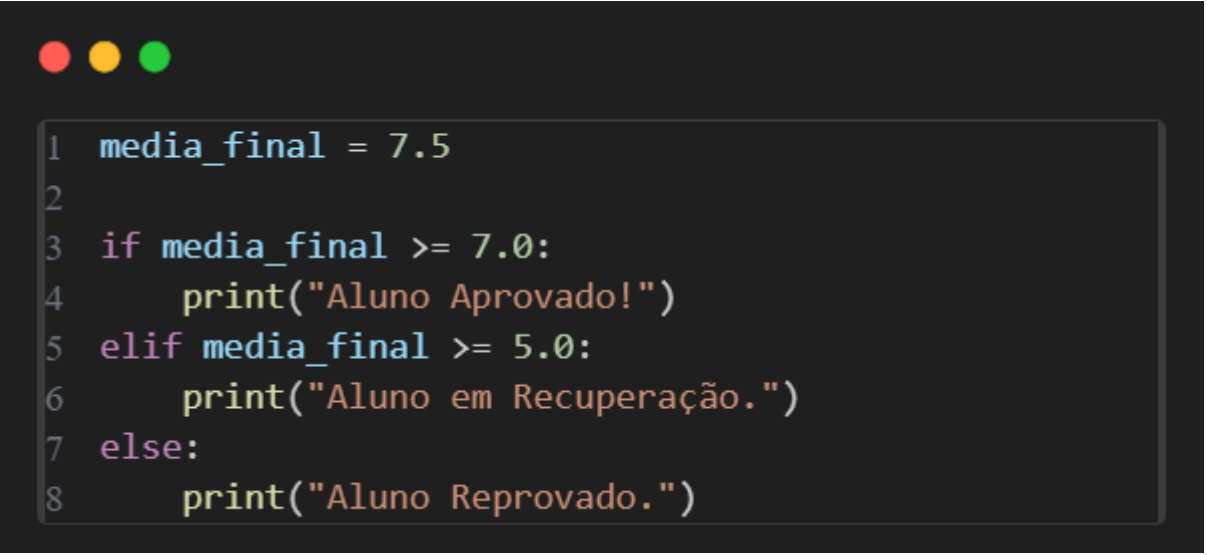
Este é o método mais antigo, herdado da linguagem C. **Não é recomendado para código novo**, mas é importante reconhecê-lo em projetos mais antigos.

Sintaxe: Usa marcadores como **%s** (string), **%d** (inteiro) e **%f** (float).

```
1 nome = "Carlos"
2 idade = 42
3 print("Nome: %s, Idade: %d" % (nome, idade))
4 # Saída: Nome: Carlos, Idade: 42
5
6 # Com formatação de float
7 print("Preço: R$ %.2f" % 19.99)
8 # Saída: Preço: R$ 19.99
9
```

6. Controle de Fluxo: Tomando Decisões (IF)

Programas precisam tomar decisões com base em condições. Para isso, usamos as estruturas `if` (se), `elif` (senão se) e `else` (senão). O código dentro de cada bloco só é executado se a sua condição for atendida.

A screenshot of a code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in Python and uses syntax highlighting: keywords like 'if', 'elif', and 'else' are in purple, strings are in orange, and the variable 'media_final' is in light blue. The code is as follows:

```
1 media_final = 7.5
2
3 if media_final >= 7.0:
4     print("Aluno Aprovado!")
5 elif media_final >= 5.0:
6     print("Aluno em Recuperação.")
7 else:
8     print("Aluno Reprovado.")
```

7. Controle de Fluxo: Repetindo Tarefas (Laços)

Para executar o mesmo bloco de código várias vezes, usamos laços. O laço **for** é geralmente usado para iterar sobre uma sequência (como uma lista). O laço **while** repete um bloco de código enquanto uma condição for verdadeira.

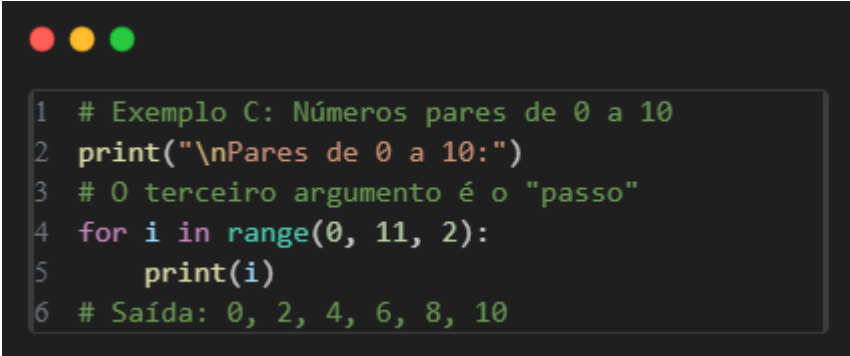
```
1 n = int(input("Digite o n termo de fibonacci: "))
2 a, b = 0, 1
3
4 # O loop executa n vezes, decrementando n a cada
  passo
5 while n > 0:
6     a, b = b, a + b
7     n -= 1
8
9 print(f"O enesimo termo de Fibonacci é: {a}")
```

```
1 n = int(input("Digite o n termo de fibonacci: "))
2 a, b = 0, 1
3 # O loop começa em 1 porque já temos o valor de fib(0)
4 # 'a' recebe o valor antigo de 'b'
5 # 'b' recebe a soma dos valores antigos de 'a' e 'b'
6 for _ in range(1, n + 1):
7     a, b = b, a + b
8 print(f"O enesimo termo de Fibonacci é: {a}")
```

Existem diversas formas de se utilizar o for então vamos nos aprofundar em mais alguns exemplos:

7.1. Iterar sobre um Intervalo de Números com `range()`

Esta é a forma mais básica, frequentemente usada quando você precisa repetir uma ação um número específico de vezes.



```
1 # Exemplo C: Números pares de 0 a 10
2 print("\nPares de 0 a 10:")
3 # O terceiro argumento é o "passo"
4 for i in range(0, 11, 2):
5     print(i)
6 # Saída: 0, 2, 4, 6, 8, 10
```

7.2. Iterar sobre uma Lista ou Tupla

Esta é a maneira mais comum e intuitiva de usar o **for**: percorrer cada item de uma lista ou tupla.

```
1 frutas = ["Maçã", "Banana", "Cereja", "Laranja"]
2 print("Minha lista de frutas:")
3 for fruta in frutas:
4     print(f"- {fruta}")
```

7.3. Iterar sobre uma String

Uma string é uma sequência de caracteres, então você pode usar um **for** para percorrer cada letra.

```
1 palavra = "Python"
2 print("Letras na palavra 'Python':")
3 for letra in palavra:
4     print(letra, end=" ") # end=" " imprime na mesma linha
5 # Saída: P y t h o n
```

7.4 . Iterar sobre um Dicionário

Você pode iterar sobre as chaves, os valores ou ambos (pares chave-valor).

```
1 aluno = {"nome": "Carlos", "idade": 25, "curso": "Engenharia"}
2
3 # Exemplo A: Iterar sobre as chaves (comportamento padrão)
4 print("Chaves do dicionário:")
5 for chave in aluno:
6     print(chave)
7 # Saída: nome, idade, curso
8
9 # Exemplo B: Iterar sobre os valores
10 print("\nValores do dicionário:")
11 for valor in aluno.values():
12     print(valor)
13 # Saída: Carlos, 25, Engenharia
14
15 # Exemplo C: Iterar sobre os pares chave-valor (mais comum)
16 print("\nItens do dicionário:")
17 for chave, valor in aluno.items():
18     print(f"{chave}: {valor}")
19 # Saída:
20 # nome: Carlos
21 # idade: 25
22 # curso: Engenharia
23
```

8.Estruturas de dados: Vetores

Quando precisamos agrupar vários valores em uma única variável, usamos estruturas de dados. Que são sequências de objetos(variáveis) alocados dinamicamente já nativo em python, o que as tornam muito mais versáteis que vetores normais em c.

Ou seja, qualquer “vetor” em python por ser alocado dinamicamente nos permite aumentar ou diminuir o vetor durante a execução, sem a necessidade de alocar.

Em python existem quatro tipos de Vetores que podem ser utilizados

- listas [**list**]
- Tuplas (**tuple**)
- Conjuntos **{set}**
- Dicionários **{dict}**

8.1. Listas (list)

Sintaxe: Elementos separados por vírgula, dentro de colchetes `[]`.

Características Principais:

- **Mutável:** Você pode alterar, adicionar e remover elementos depois que a lista é criada.
- **Ordenada:** Os elementos mantêm a ordem em que foram adicionados.
- **Indexada:** Você pode acessar qualquer elemento pelo seu índice numérico (começando em 0).
- **Permite Duplicados:** Uma lista pode conter elementos repetidos sem problemas

```
1 # Criando uma lista de compras
2 compras = ["maçã", "banana", "leite", "pão"]
3
4 # Acessando itens pelo índice (começa em 0)
5 print(f"O primeiro item é: {compras[0]}")
6
7 # Adicionando um item ao final
8 compras.append("queijo")
9 print(f"Lista com novo item: {compras}")
10
11 # Removendo o último item
12 compras.pop()
13 # ou pop(0) para remover o primeiro item
14 print(f"Lista após remover item: {compras}")
```

8.2. Tuplas (tuple)

Uma tupla é muito parecida com uma lista, mas com uma diferença crucial: ela é **imutável**. Uma vez criada, uma tupla não pode ser alterada.

- **Sintaxe:** Elementos separados por vírgula, dentro de parênteses `()`.
- **Características Principais:**
 - **Imutável:** Não se pode adicionar, remover ou alterar elementos.
 - **Ordenada:** Assim como as listas, a ordem dos elementos é preservada.
 - **Indexada:** Os elementos podem ser acessados por seu índice numérico.
 - **Permite Duplicados:** Também pode conter elementos repetidos.

```
1 # Criando uma tupla para coordenadas RGB
2 cor_primaria = (255, 0, 0) # Vermelho
3
4 # Acessando um item
5 print(cor_primaria[0]) # Saída: 255
6
7 # Tentando modificar (isto causará um erro!)
8 # cor_primaria[0] = 100 # TypeError: 'tuple' object
  # does not support item assignment
```

8.3. Conjuntos (set)

Um conjunto é uma coleção de itens **únicos** e **não ordenada**. Sua principal utilidade é verificar rapidamente se um item pertence à coleção e remover elementos duplicados.

- **Sintaxe:** Elementos separados por vírgula, dentro de chaves `{}` (sem pares chave-valor). Para criar um conjunto vazio, use `set()`.
- **Características Principais:**
 - **Mutável:** Você pode adicionar e remover elementos.
 - **Não Ordenado:** Não há garantia sobre a ordem dos elementos.
 - **Não Indexado:** Você não pode acessar elementos por um índice.
 - **Não Permite Duplicados:** Se você adicionar um item que já existe, nada acontece.

```
1 # Criando um conjunto de tags
2 tags = {"python", "dados", "web", "python"}
3
4 print(tags) # Saída: {'web', 'python', 'dados'}
5 #(a ordem pode variar e "python" aparece só uma vez)
6
7 # Verificando se um item existe (muito rápido!)
8 print("python" in tags) # Saída: True
9
10 # Adicionando um novo item
11 tags.add("ia")
12 print(tags) # Saída: {'web', 'python', 'ia', 'dados'}
13
14 # Removendo duplicados de uma lista
15 numeros = [1, 2, 3, 2, 4, 1, 5]
16 numeros_unicos = set(numeros)
17 print(numeros_unicos) # Saída: {1, 2, 3, 4, 5}
```

8.4. Dicionários (dict)

Diferente de listas e tuplas, os dicionários não armazenam elementos em uma sequência ordenada, mas sim como um conjunto de pares **chave-valor**. Pense neles como uma agenda de contatos, onde o nome é a "chave" e o telefone é o "valor".

- **Sintaxe:** Pares **chave: valor** separados por vírgula, dentro de chaves `{}`.
- **Características Principais:**
 - **Mutável:** Você pode adicionar, alterar e remover pares chave-valor.
 - **Ordenado (a partir do Python 3.7):** As versões modernas do Python mantêm a ordem de inserção das chaves.
 - **Acessado por Chave:** Os valores são acessados através de suas chaves únicas, não por um índice numérico.
 - **Chaves Únicas:** Não pode haver chaves duplicadas em um dicionário.

```
1 # Criando um dicionário para um aluno
2 aluno = {
3     "nome": "Carlos",
4     "idade": 22,
5     "curso": "Engenharia"
6 }
7
8 # Acessando um valor pela sua chave
9 print(f"O nome do aluno é: {aluno['nome']}")
10
11 # Adicionando um novo par chave-valor
12 aluno["cidade"] = "Rio de Janeiro"
13 print(f"Dicionário atualizado: {aluno}")
```